Combinatorial Algorithms (Math 482), 2026 Spring Problem Set 5

Due by Wednesday, February 25, 2026.

1. **Shared Kanpsack.** Alice and Bob go for a hike together with one (shared) backpack of capacity C. There are n possible items they consider taking for the hike. However, some items are more valuable for Alice, and some are more valuable for Bob. The n items have values $U[1..n]$ for Alice, values $V[1..n]$ for Bob, and weights $W[1..n]$. If $B[1..n]$ is the binary array that indicates which items they take, they would like to maximize the objective function
$$\min\Big(\sum_{i=1}^{n} B[i]\cdot U[i], \sum_{i=1}^{n} B[i]\cdot V[i]\Big),$$
subject to the weight constraint $\sum_{i=1}^{n} B[i]\cdot W[i] \le C$.

   Design a dynamic programming algorithm for this problem: Define subproblems, find a recurrence relation, and write a pseudo-code that finds the maximum value of the objective function. What is the running time of your algorithm?

2. **Longest Rapidly Increasing Subsequence.** Given an array $A[1..n]$ of positive integers, we say that a subsequence
$$(A[i_1], A[i_2], \ldots, A[i_k])$$
is *rapidly increasing* if $i_1 < i_2 < \ldots < i_k$ and $2\cdot A[i_j]+3 < A[i_{j+1}]$ for all $j = 1, \ldots, k-1$.

   Design a dynamic programming algorithm for finding the longest rapidly increasing subsequence in a given array $A[1..n]$. Define subproblems, find a recurrence relation, and write a pseudo-code that returns the maximum length of a rapidly increasing subsequence. What is the running time of your algorithm?

3. **Dynamic Programming tables.** In each case, we are given a recursion for a two-dimensional array $A[1..n, 1..n]$. Either write two for-loops that compute all values of the array; or prove that it is impossible to compute all values of the array using the formula.

   (a)
$$A[i,j] = \begin{cases} i+j & \text{if } i=1 \text{ or } j=n \\ A[i-1,j] + A[i,j+1]) & \text{otherwise.} \end{cases}$$

   (b)
$$A[i,j] = \begin{cases} i+j & \text{if } i=1,\ i=2,\ j=1,\ \text{or } j=n \\ A[i-2,j+1] + A[i+1,j-1] & \text{otherwise.} \end{cases}$$

   (c)
$$A[i,j] = \begin{cases} i+j & \text{if } i=1,\ i=n,\ \text{or } j=1 \\ A[i-1,j] + A[i-1,j-1] + A[i+1,j], & \text{otherwise.} \end{cases}$$